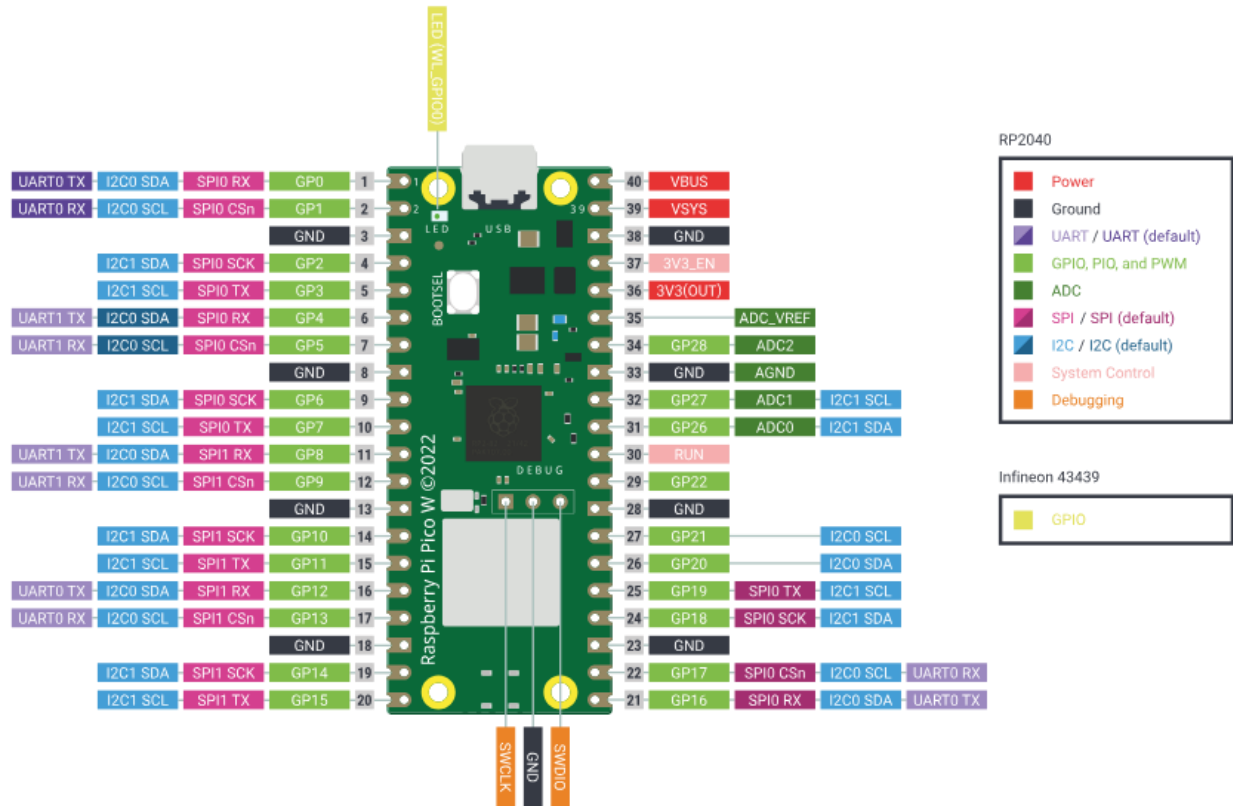


PICO-SDK INSTALLATION AND COMPILATION USING RASPBERRY PI-PICO-W SERIES



GUIDED BY:
DURAI P S

MENTOR:
DANIEL RAJ A

TEAM:
1. KARTHICK A
2. LUCKSHANA V R
3. SHREE SUBI G
4. JANASRUTHIKA P R

TABLE OF CONTENT

S.no	Content	Page number
1.	<u>Cloning SDK and examples to the directory</u>	<u>1</u>
2.	<u>Create project folder & export SDK path</u>	<u>4</u>
3.	<u>Build the project using CMake and Make</u>	<u>5</u>
4.	<u>Copy .uf2 file to flash the microcontroller</u>	<u>5</u>
5.	<u>Troubleshooting techniques</u>	<u>6</u>
6.	<u>Revision history</u>	<u>8</u>
7.	Appendix	9

LED BLINKING THROUGH TERMINAL

1. CLONING SDK AND EXAMPLES TO THE DIRECTORY:

- a) To clone the SDK, Create a new directory and clone the SDK as given below

```
$ mkdir pico
$ cd pico
$ git clone https://github.com/raspberrypi/pico-sdk.git --branch master
$ cd pico-sdk
$ git submodule update --init
```

- b) Now clone the examples into pico directory.

```
$ cd ..
$ git clone https://github.com/raspberrypi/pico-examples.git --branch master
$ git clone https://github.com/raspberrypi/pico-examples
```

- c) To build applications we need CMake, ARM GNU Toolchain. To install those, run the command.

```
$ sudo apt update
$ sudo apt install cmake gcc-arm-none-eabi libnewlib-arm-none-eabi build-essential
```

- **Cmake** = CMake is an open-source, cross-platform tool used for managing the build process of software projects. It allows developers to define the build process in a platform-independent way by creating configuration files (CMakeLists.txt)
- **Gcc-arm-none-eabi** = Compiling C/C++ code for ARM Cortex-M microcontrollers.
Bare-metal development, meaning there's no OS — you directly program the hardware.
- **libnewlib-arm-none-eabi** = A C standard library (like stdio.h, math.h) optimized for embedded systems.
- **build-essential** = A meta-package that installs tools like make, g++ and other basic development tools needed to compile C/C++ code.

2. CREATE PROJECT FOLDER AND EXPORT SDK PATH:

- d) When doing a new project, check if the SDK is up to date.

```
$ cd pico-sdk  
$ git pull  
$ git submodule update
```

- e) Get the blink code which is saved in the pico-examples, which we cloned in the step 1 and create a “build” directory to build the program in the pico-examples folder.

```
$ cd ..  
$ cd pico-examples  
$ mkdir build  
$ cd build
```

- f) Then, set the PICO_SDK_PATH, assuming you cloned the pico-sdk and pico-examples repositories into the same directory:

```
$ export PICO_SDK_PATH=../pico-sdk
```

- g) Replace ../../ with your path in which pico-sdk is present.

Example:

```
$ export PICO_SDK_PATH=/home/luckshana/new/pico/pico-sdk.
```

3. BUILD THE PROJECT USING CMAKE AND MAKE:

- h) Compile the program.

```
$ cmake ..  
$ cd blink  
$ make
```

- i) If cmake file shows any error, update the CMake version after checking the version using.

```
cmake --version
```

```
$ sudo apt remove cmake  
$ sudo snap install cmake --classic
```

4. COPY .UF2 FILE TO FLASH THE MICROCONTROLLER:

- j) After the completion of progress, copy the uf2 file.

```
luckshana@luckshana-Inspiron-15-3511:~/new/pico/pico-examples/build/blink$ ls  
blink.bin  blink.elf      blink.hex  CMakeFiles      Makefile  
blink.dis  blink.elf.map  blink.uf2  cmake_install.cmake
```

- k) Copy the blink.uf2 file into the Pico folder (RPI-RP2).

```
luckshana@luckshana-Inspiron-15-3511:~/new/pico/pico-examples/build/blink$ cp blink.uf2 /media/luckshana/RPI-RP2/  
luckshana@luckshana-Inspiron-15-3511:~/new/pico/pico-examples/build/blink$
```

```
$ cp blink.uf2 /media/.../RPI-RP2
```

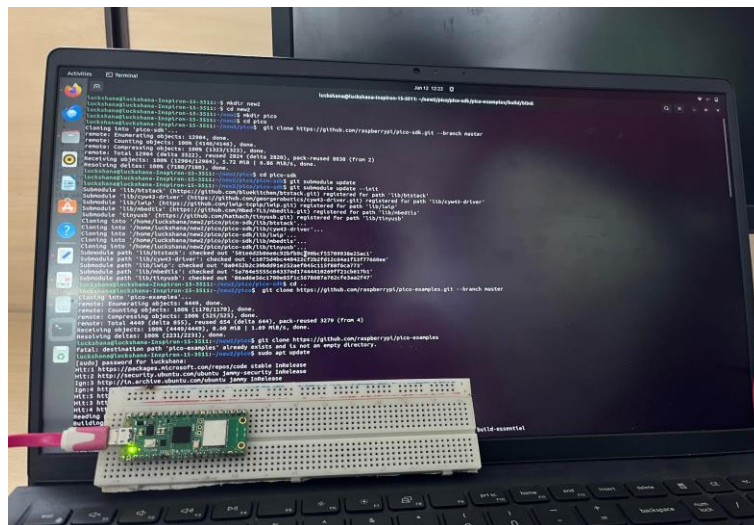


Fig.1: Inbuilt LED blinking

5. TROUBLESHOOTING TECHNIQUES:

- Even after copying the file if the LED is not blinking, check the blink.c file. Open it using the vim or nano editor.
- Check the code and change if there are any errors

CODE FOR BLINKING (pico W):

```
#include "pico/stdlib.h"
#include "pico/cyw43_arch.h" // Required for Pico W LED control

#ifndef LED_DELAY_MS
#define LED_DELAY_MS 250
#endif

int main() {
```

```

// Initialize the Wi-Fi chip (needed to access onboard LED on Pico W)
if (cyw43_arch_init()) {
    return -1; // Initialization failed
}

while (true) {
    // Turn LED on
    cyw43_arch_gpio_put(CYW43_WL_GPIO_LED_PIN, 1);
    sleep_ms(LED_DELAY_MS);

    // Turn LED off
    cyw43_arch_gpio_put(CYW43_WL_GPIO_LED_PIN, 0);
    sleep_ms(LED_DELAY_MS);
}
}

```

- a) If we change the code, we must make changes in the CMakeLists.txt files too.

```

add_executable(blink
    blink.c
)

# pull in common dependencies
target_link_libraries(blink pico_stdlib)

if (PICO_CYW43_SUPPORTED)
    target_link_libraries(blink pico_stdlib pico_cyw43_arch_none)
endif()

# create map/bin/hex file etc.
pico_add_extra_outputs(blink)

# add url via pico_set_program_url
example_auto_set_url(blink)

```

Link with the correct libraries in your **CMakeLists.txt**

```

Scanning dependencies of target bs2_default
Built target bs2_default
Scanning dependencies of target bs2_default_library
Built target bs2_default_library
Scanning dependencies of target blink
Building C object blink/CMakeFiles/blink.dir/blink.c.o
/home/luckshana/new2/pico/pico-examples/blink/blink.c:5:10: fatal error: pico/cyw43_arch.h: No such file or direct
5 | #include "pico/cyw43_arch.h" // Required for Pico W LED control
|
compilation terminated.
make[2]: *** [blink/CMakeFiles/blink.dir/build.make:76: blink/CMakeFiles/blink.dir/blink.c.o] Error 1
make[1]: *** [CMakeFiles/Makefile2:5498: blink/CMakeFiles/blink.dir/all] Error 2
make: *** [Makefile:91: all] Error 2

```

b) If the above error arises type the following command inside the build directory.

```
$ cmake .. -DPICO_BOARD=pico_w
```

6. REVISION HISTORY:

Rev no.	Date	Section	Change Description	Modified By
0.01	12.06.2025	All	Initial contents	Luckshana VR
0.02	13.06.2025	Error on Cmake update	Changes in Step 6	Shree Subi G
0.02	13.06.2025	Code section CMakeLists.txt file	Changes in blink.c file Changes in target file	Karthick A
0.03	13.06.2025	Directory path	Change the flow of path	Janasruthika P R

7. APPENDIX:

REFERENCE LIST:

- 1) <https://www.youtube.com/watch?v=JhajoAyP8e4>
- 2) <https://github.com/raspberrypi/pico-examples>
- 3) <https://www.raspberrypi.com/documentation/microcontrollers/pico-series.html>
- 4) <https://datasheets.raspberrypi.com/picow/pico-w-datasheet.pdf>